# ARM Internship Prep ��? 1-Month Sprint Roadmap

# ARM Internship Prep — 1-Month Sprint Roadmap

**Built for:** Intermediate C/C++, new to embedded/ARM specifically · ~56 hrs/week (8 hrs × 7 days) · Goal: ARM internship-ready portfolio + CV, self-test sprint, no hard deadline · Toolchain: STM32 + Arm GNU toolchain + FreeRTOS + GDB/OpenOCD + Ceedling

## Quick Orientation

This compresses your original 6-month plan into 4 weeks by cutting scope, not depth. You're skipping the C-fundamentals month (you don't need it) and merging two of the five original project ideas into one flagship project instead of building all five shallowly. The trade-off: you'll ship *one* excellent, well-tested, well-documented firmware project plus a tight CV — not a broad tour of every embedded topic. That's actually closer to what a recruiter wants anyway: your original doc's own framing was "turn skills into evidence," and one deep project is stronger evidence than five thin ones.

One honest flag: literal 8h × 7 days with zero slack is a real burnout/slippage risk — embedded debugging in particular eats time unpredictably (a "20 minute" wiring bug can eat an afternoon). Each week below has one lighter day built in for catch-up so the 7-day cadence survives contact with reality.

## Core Concepts You Must Actually Understand

- **Cortex-M architecture & memory map** — you can't reason about stack overflows, linker scripts, or footprint without knowing where flash/RAM/peripherals actually live.
- **Interrupts vs. DMA vs. RTOS tasks** — this is the single most common "sounds fine, is actually broken" mistake in embedded C. If you don't deeply get why heavy work in an ISR is dangerous, you'll ship code that works in the demo and deadlocks under load.
- **RTOS primitives (tasks, queues, mutexes/semaphores, priority)** — race conditions and priority inversion are invisible until they aren't. This is exactly the kind of judgment ARM's postings flag as a differentiator.
- **Debugging methodology (GDB/OpenOCD, fault status/address registers)** — this is the actual skill being tested in interviews. "I can attach a debugger and find a HardFault cause" is worth more than any feature you build.
- **ELF/linker basics (readelf/objdump, memory layout)** — needed to talk credibly about footprint, and it's what "systems thinking" questions probe.
- **Git workflow discipline** — branches, PRs, structured commits. Cheap to learn, expensive to fake convincingly in an interview.

## Tedious Parts You Can Lean on AI/Tooling For

- **STM32CubeMX pin/clock/peripheral config** — let it generate init code; skim the output once to confirm it matches your intent, don't hand-write it.
- **HAL driver boilerplate** — use ST's HAL where it's not the point of the exercise; only hand-roll the parts that are actually the "flagship" logic (ring buffer, task design, fault capture).
- **Ceedling YAML scaffolding** — copy from the official quickstart, don't memorize the config format.
- **CLI/UART menu parsing boilerplate** — fine to draft with AI assistance and adapt; the interesting part is the state machine behind it, not the string parsing.

## Learning Path

### Phase 1: Foundations — Toolchain, Architecture, Debugging (Week 1, ~56 hrs)

**Concepts covered:** Arm GNU toolchain + build system, Cortex-M architecture basics, GDB + OpenOCD workflow, memory map, Git branching discipline.

**Daily rhythm (rough split across the ~8h):** - 1.5h reading: Cortex-M architecture reference + relevant ARM docs section for the day - 4.5h hands-on: toolchain setup, CMake build, first peripheral bring-up, debug sessions - 1.5h: write up what broke and how you fixed it (this becomes your "debug cookbook") - Day 7 (lighter): catch-up + review; can you explain the memory map out loud without notes?

**Resources:** - Docs: Arm GNU Toolchain (developer.arm.com/downloads/-/arm-gnu-toolchain-downloads) - Docs: GDB official manual (sourceware.org/gdb/current/onlinedocs/gdb) - Docs: OpenOCD user guide (openocd.org/doc/pdf/openocd.pdf) - Tooling: STM32CubeIDE / STM32CubeMX (st.com) - Reference: Pro Git (git-scm.com/book/en/v2) — read the branching/rebasing chapters, not the whole book

**Checkpoint project:** A minimal, reusable firmware scaffold: CMake build, UART logging, one GPIO peripheral bring-up, committed with clean history. Do one deliberate debug session — set a breakpoint, inspect a register and a memory address, step through reset — and write down exactly what you did. This scaffold is the base every later phase builds on, so don't rush it.

### Phase 2: RTOS + Flagship Implementation Sprint 1 (Week 2, ~56 hrs)

**Concepts covered:** FreeRTOS tasks/queues/semaphores, interrupt-safe design, DMA-based UART TX, ring buffers with backpressure.

**Daily rhythm:** - 1h reading: FreeRTOS docs section for the day (queues/mutexes/semaphores, then task priority) - 5h hands-on: implement one feature end-to-end per day (sampling task → DMA UART TX → ring buffer → CLI) - 1.5h: measure something (jitter, buffer

drop rate) and log it - Day 7 (lighter): integration day — get all the week's pieces talking to each other; this is usually where the real bugs show up

**Resources:** - Docs: FreeRTOS official docs — queues, mutexes, semaphores (freertos.org/Documentation) - Docs: CMSIS-RTOS2 overview (arm-software.github.io/CMSIS_6/main/RTOS2)

**Checkpoint project:** Real-time sensor node — separate FreeRTOS tasks for sampling, DMA-driven UART logging (ring buffer + backpressure), and a basic CLI, with sensor input either real or simulated. By end of week: stable operation under stress with no data loss, and you can state your jitter numbers.

### Phase 3: Fault Handling + Testing + Instrumentation (Week 3, ~56 hrs)

**Concepts covered:** Cortex-M fault status/address registers, crash capture + symbolication, host-run unit testing (Ceedling/Unity/CMock), CI, footprint measurement.

**Daily rhythm:** - 1h reading: fault register docs / Ceedling docs as needed - 4.5h hands-on: fault handler implementation, host unit test suite, CI setup - 2h: fault injection testing + write the "how to debug a crash in under 10 minutes" triage guide - Day 7 (lighter): coverage review, footprint measurement (size/readelf), buffer for slippage

**Resources:** - Docs: Fault status and fault address registers (developer.arm.com/documentation) - Docs: FreeRTOS — Debugging Hard Faults on Cortex-M (freertos.org) - Tooling: Ceedling + Unity + CMock (github.com/throwtheswitch/ceedling, throwtheswitch.org/cmock) - Docs: objdump / readelf (sourceware.org/binutils/docs)

**Checkpoint project:** Fault handler bolted onto the sensor node from Phase 2 — captures register/stack context on HardFault, stores a crash report, and a host script symbolicates it back to function/line. Plus: a host-run unit test suite (ring buffer, parser, config logic) running green in CI, with a documented footprint (flash/RAM usage) measurement.

### Phase 4: Polish, Documentation, CV, Interview Prep (Week 4, ~56 hrs)

**Concepts covered:** design-doc writing, release packaging, CV bullet construction, interview practice across coding/embedded/systems threads.

**Daily rhythm:** - 2h: repo polish — README, design doc (requirements → architecture → interfaces), tagged release - 2h: CV — draft using the structure and bullet templates from your original doc (Section: "CV guidance"), swap in your *actual* measured numbers (jitter, drop rate, footprint) rather than placeholders - 3h: interview prep — rotate across coding (C/C++ memory safety, ring buffers, bit manipulation), embedded (mutex vs. semaphore, ISR deferral, RTOS priority design), and systems (fault registers, ELF inspection, ABI basics) - 1h: demo — record or script a short walkthrough of the project - Day 7 (lighter): mock interview with a friend/mentor if possible, or talk through your project out loud to yourself as if presenting it — do this even if it feels silly, it exposes gaps fast

**Checkpoint project:** No new project — the deliverable this week is a tagged release of the flagship repo (clean README, design doc, demo script) plus a finished one-page CV with tailored bullets backed by real numbers from your own project.

## Project Ladder

1. **Firmware scaffold** — reusable CMake build + UART logging + GPIO bring-up + one documented debug session (Week 1)
2. **Real-time sensor node** — FreeRTOS tasks, DMA UART logging with ring buffer, basic CLI, measured jitter (Week 2)
3. **Fault monitor add-on + test suite** — crash capture/symbolication bolted onto the sensor node, host unit tests running in CI, footprint measurement (Week 3)
4. **Release + CV** — polished repo, design doc, demo, one-page CV (Week 4)

## Anti-Tutorial-Hell Rules

- For every hour spent reading docs or watching a walkthrough, spend at least 3 hours building without it open — you have almost no slack this month for passive consumption.
- Self-test before moving to the next phase: can you explain, out loud, *why* the previous phase's feature is designed the way it is (not just how it works)? If not, you're not ready to build on it yet.
- Docs are fine to reopen as a reference lookup mid-build. If you're copy-pasting a solution rather than adapting something you understand, the checkpoint doesn't count as done — go back and rebuild the piece from a blank file.

## Resource List

**Docs & primary references:** - Arm GNU Toolchain — developer.arm.com/downloads/-/arm-gnu-toolchain-downloads - GDB manual — sourceware.org/gdb/current/onlinedocs/gdb - OpenOCD user guide — openocd.org/doc/pdf/openocd.pdf - FreeRTOS docs (queues/mutexes/semaphores, hard fault debugging) — freertos.org - CMSIS-RTOS2 — arm-software.github.io/CMSIS_6/main/RTOS2 - ARM fault status/address register docs — developer.arm.com/documentation - Ceedling / CMock — github.com/throwtheswitch/ceedling, throwtheswitch.org/cmock - objdump / readelf — sourceware.org/binutils/docs - Pro Git — git-scm.com/book/en/v2 - C++ Core Guidelines / clang-tidy — isocpp.github.io/CppCoreGuidelines, clang.llvm.org/extra/clang-tidy

**Tooling:** - STM32CubeIDE / STM32CubeMX — st.com

## Suggested Weekly Rhythm

Roughly, per 8-hour day: 1–1.5h focused reading/spec-reading, 4.5–5h hands-on building, 1–2h debugging/testing/measuring, and a short daily note on what broke and why. Keep day 7 of each week deliberately lighter — integration, catch-up, and self-testing rather than new material — so a single bad debugging day doesn't blow up the whole week's plan.